

Assignment 2: Coding Basics

Isabelle Kuehn

OVERVIEW

This exercise accompanies the lessons/labs in Environmental Data Analytics on coding basics.

Directions

1. Rename this file `<FirstLast>_A02_CodingBasics.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.
6. After Knitting, submit the completed exercise (PDF file) to Canvas.

Basics, Part 1

1. Generate a sequence of numbers from one to 55, increasing by fives. Assign this sequence a name.
2. Compute the mean and median of this sequence.
3. Ask R to determine whether the mean is greater than the median.
4. Insert comments in your code to describe what you are doing.

```
#1.  
seq_numbers <- seq(1,55,5)  
seq_numbers
```

```
## [1] 1 6 11 16 21 26 31 36 41 46 51
```

```
#2.  
mean_data <- mean(seq_numbers)  
mean_data
```

```
## [1] 26
```

```
median_data <- median(seq_numbers)  
median_data
```

```
## [1] 26
```

```
#3.
determine_stats <- function(mean_data, median_data) {
  if(mean_data > median_data) {
    return("The mean is greater than the median")
  }
  if(median_data > mean_data) {
    return("The median is greater than the mean")
  }
  if(mean_data == median_data) {
    return("The median is equal to the mean")
  }
}
determine_stats(mean_data, median_data)
```

```
## [1] "The median is equal to the mean"
```

Basics, Part 2

5. Create three vectors, each with four components, consisting of (a) student names, (b) test scores, and (c) whether they are on scholarship or not (TRUE or FALSE).

```
vector_a <- c("Chester", "Isabelle", "Alister", "Elliot")
vector_b <- c(73, 82, 98, 89)
vector_c <- c(FALSE, FALSE, TRUE, TRUE)
```

6. Label each vector with a comment on what type of vector it is.

```
v_names <- vector_a      #character vector for student names
v_scores <- vector_b     #number vector for test scores
v_schol <- vector_c      #logical vector for scholarship
```

7. Combine each of the vectors into a data frame. Assign the data frame an informative name.

```
student_info <- data.frame(v_names, v_scores, v_schol)
```

8. Label the columns of your data frame with informative titles.

```
names(student_info) <- c("Student Names", "Test Scores", "Scholorship T/F")
print(student_info)
```

```
##   Student Names Test Scores Scholarship T/F
## 1      Chester         73          FALSE
## 2     Isabelle         82          FALSE
## 3      Alister         98           TRUE
## 4       Elliot         89           TRUE
```

9. QUESTION: How is this data frame different from a matrix?

Answer: This data frame is different from a matrix because data frames store vectors of equal length where columns in the data set can be defined as separate modes. This means data frames can use numeric and categorical data, while matrices are less generalized and must contain elements of the same type.

10. Create a function with one input. In this function, use `if...else` to evaluate the value of the input: if it is greater than 50, print the word "Pass"; otherwise print the word "Fail".
11. Create a second function that does the exact same thing as the previous one but uses `ifelse()` instead of `if...else`.
12. Run both functions using the value 52.5 as the input
13. Run both functions using the **vector** of student test scores you created as the input. (Only one will work properly...)

#10. Create a function using if...else

```
x <- 42
compare <- function(x) {
  if(x > 50){
    return("Pass")
  }
  else {
    return("Fail")
  }
}
compare(x)
```

```
## [1] "Fail"
```

#11. Create a function using ifelse()

```
compare_2 <- function(x) {
  ifelse(x > 50, "Pass", "Fail")
}
compare_2(x)
```

```
## [1] "Fail"
```

#12a. Run the first function with the value 52.5

```
compare(52.5)
```

```
## [1] "Pass"
```

#12b. Run the second function with the value 52.5

```
compare_2(52.5)
```

```
## [1] "Pass"
```

#13a. Run the first function with the vector of test scores

compare_ages(v_scores) #cannot run because of vector input (commented out for pdf)

```
for(i in v_scores) {
  print(compare(i)) #function put in for loop works because it iterates over each score!
}
```

```
## [1] "Pass"
## [1] "Pass"
## [1] "Pass"
## [1] "Pass"
```

```
#13b. Run the second function with the vector of test scores
compare_2(v_scores)
```

```
## [1] "Pass" "Pass" "Pass" "Pass"
```

14. QUESTION: Which option of `if...else` vs. `ifelse` worked? Why? (Hint: search the web for “R vectorization”)

Answer: `ifelse()` worked because it is essentially a vectorization of the `if...else()` statement. Initially, using `if...else` failed because a vector contains multiple values which makes its length greater than 1. This means it would rely on a loop, like the `for` loop used, to run each component of the vector against the condition. This process is less efficient and leaves the code open to more errors. Using `ifelse()` allows the function to run through every value in the vector at once, removing the need for a loop and creating more efficient, less error-prone code.

NOTE Before knitting, you’ll need to comment out the call to the function in Q13 that does not work. (A document can’t knit if the code it contains causes an error!)